

Memory Hierarchy (III)

Outline

- Cache Memory
- Write Cache Friendly codes
- Matrix multiplication
- Memory mountain
- Suggested Reading: 6.4, 6.5, 6.6

Issues with Writes

- Write hits
 - Write through
 - Cache updates its copy
 - Immediately writes the corresponding cache block to memory
 - Write back
 - Defers the memory update as long as possible
 - Writing the updated block to memory only when it is evicted from the cache
 - Maintains a **dirty bit** for each cache line

Issues with Writes

- Write misses
 - Write-allocate
 - Loads the corresponding memory block into the cache
 - Then updates the cache block
 - No-write-allocate
 - Bypasses the cache
 - Writes the word directly to memory
- Combination
 - Write through, no-write-allocate
 - Write back, write-allocate (modern implementation)

Core 0

L1 d-cache, i-cache
32k 8-way
Access: 4 cycles

Regs

L1
d-cache

L1
i-cache

L2 unified cache

L2 unified-cache
256k 8-way
Access: 10 cycles

Regs

L1
d-cache

L1
i-cache

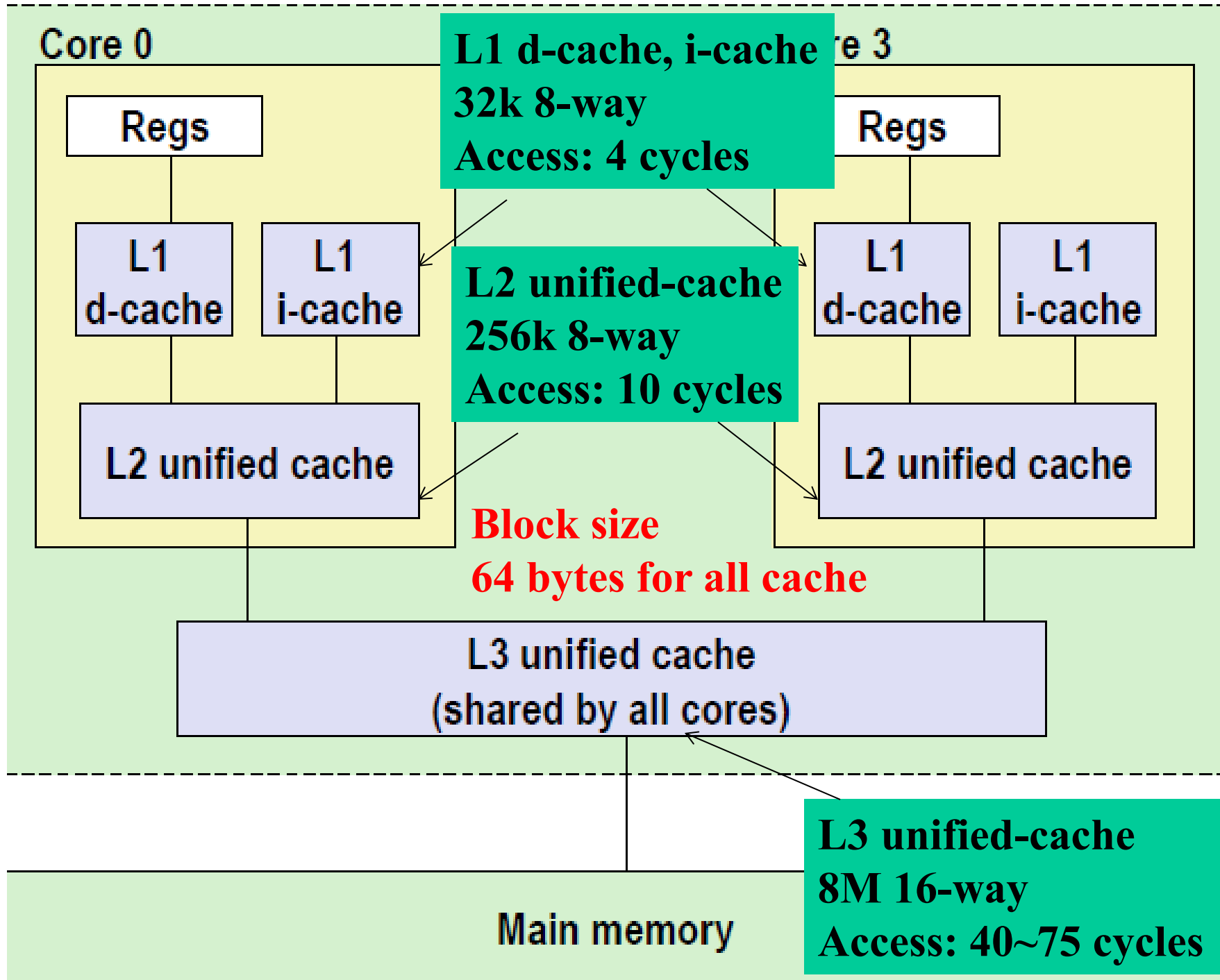
L2 unified cache

Block size
64 bytes for all cache

L3 unified cache
(shared by all cores)

L3 unified-cache
8M 16-way
Access: 40~75 cycles

Main memory



Cache performance metrics

- Miss Rate
 - fraction of memory references not found in cache (misses/references)
 - Typical numbers:
 - 3-10% for L1
 - Can be quite small (<1%) for L2, depending on size
- Hit Rate
 - fraction of memory references found in cache (1 - miss rate)

Cache performance metrics

- Hit Time
 - time to deliver a line in the cache to the processor (includes time to determine whether the line is in the cache)
 - Typical numbers:
 - 4 clock cycles for L1 in core i7
 - 11 clock cycles for L2 in core i7
- Miss Penalty
 - additional time required because of a miss
 - Typically 50-200 cycles for main memory (Trend: increasing!)

What does hit rate mean?

- Consider
 - Hit Time: 2 cycles
 - Miss Penalty: 200 cycles
 - Average access time:
 - Hit rate 99%: $2 * 0.99 + 200 * 0.01 = 4$ cycles
 - Hit rate 97%: $2 * 0.97 + 200 * 0.03 = 8$ cycles
- This is why "miss rate" is used instead of "hit rate"

Cache performance metrics

- Cache size
 - Hit rate vs. hit time
- Block size
 - Spatial locality vs. temporal locality
- Associativity
 - Thrashing, cost, speed, and miss penalty
- Write strategy
 - Simple, read misses, fewer transfer

Writing cache-friendly code

- Principles
 - Programs with better **locality** will tend to have lower miss rates
 - Programs with lower miss rates will tend to run faster than programs with higher miss rates

Writing cache-friendly code

- Basic approach
 - Make the common case go fast
 - Programs often spend most of their time in a few core functions
 - These functions often spend most of their time in a few loops
 - Minimize the number of cache misses in each inner loop

Writing cache-friendly code

```
int sumvec(int v[N])
```

```
{
```

```
    int i, sum = 0 ;
```

← Temporal locality,

These variables are usually put in registers

```
    for (i = 0 ; i < N ; i++)
```

```
        sum += v[i] ;
```

```
    return sum ;
```

```
}
```

v[i]	i=0	i= 1	i= 2	i= 3	i= 4	i= 5	i= 6	i= 7
Access order, [h]it or [m]iss	1[m]	2[h]	3[h]	4[h]	5[m]	6[h]	7[h]	8[h]

Writing cache-friendly code

- Temporal locality
 - Repeated references to local variables are good because the compiler can cache them in the register file

Writing cache-friendly code

- Spatial locality
 - Stride-1 references patterns are good because caches at all levels of the memory hierarchy store data as contiguous blocks
- Spatial locality is especially important in programs that operate on multidimensional arrays

Writing cache-friendly code

Example (M=4, N=8)

```
int sumarrayrows(int a[M][N])
{
    int i, j, sum = 0 ;

    for ( i = 0 ; i < M ; i++ )
        for ( j = 0 ; j < N ; j++ )
            sum += a[i][j] ;
    return sum ;
}
```

Writing cache-friendly code

a[i][j]	j=0	j= 1	j= 2	j= 3	j= 4	j= 5	j= 6	j= 7
i=0	1[m]	2[h]	3[h]	4[h]	5[m]	6[h]	7[h]	8[h]
i=1	9[m]	10[h]	11[h]	12[h]	13[m]	14[h]	15[h]	16[h]
i=2	17[m]	18[h]	19[h]	20[h]	21[m]	22[h]	23[h]	24[h]
i=3	25[m]	26[h]	27[h]	28[h]	29[m]	30[h]	31[h]	32[h]

Writing cache-friendly code

Example (M=4, N=8)

```
int sumarraycols(int a[M][N])
{
    int i, j, sum = 0 ;

    for ( j = 0 ; j < N ; j++ )
        for ( i = 0 ; i < M ; i++ )
            sum += a[i][j] ;
    return sum ;
}
```

Writing cache-friendly code

a[i][j]	j=0	j= 1	j= 2	j= 3	j= 4	j= 5	j= 6	j= 7
i=0	1[m]	5[m]	9[m]	13[m]	17[m]	21[m]	25[m]	29[m]
i=1	2[m]	6[m]	10[m]	14[m]	18[m]	22[m]	26[m]	30[m]
i=2	3[m]	7[m]	11[m]	15[m]	19[m]	23[m]	27[m]	31[m]
i=3	4[m]	8[m]	12[m]	16[m]	20[m]	24[m]	28[m]	32[m]

Matrix Multiplication

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$

$$c_{22} = a_{21}b_{12} + a_{22}b_{22}$$

Matrix Multiplication Implementation

```
/* ijk */  
for (i=0; i<n; i++) {  
    for (j=0; j<n; j++) {  
        c[i][j] = 0.0;  
        for (k=0; k<n; k++)  
            c[i][j] += a[i][k] * b[k][j];  
    }  
}
```

$O(n^3)$ adds and multiplies

Each n^2 elements of A and B is read n times

Matrix Multiplication

- Assumptions:
 - Each array is an $n \times n$ array of double, with size 8
 - There is a single cache with a 32-byte block size ($B=32$)
 - The array size n is so large that a single matrix row does not fit in the cache
 - The compiler stores local variables in registers, and thus references to local variables inside loops do not require any load and store instructions

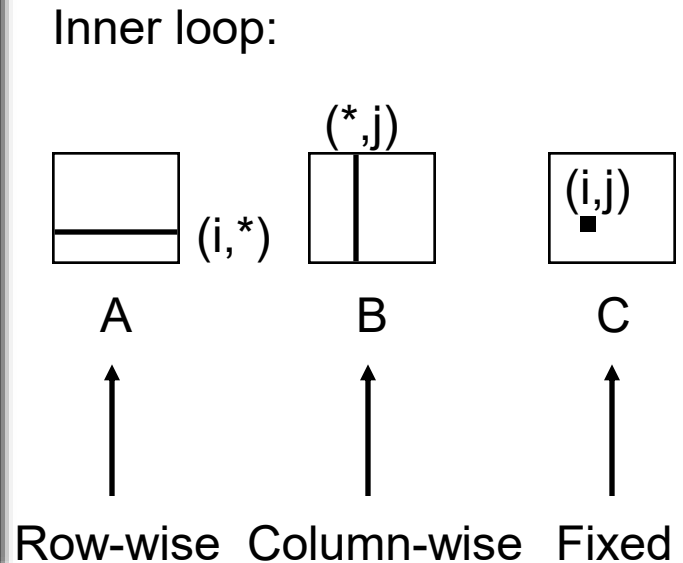
Matrix Multiplication

```
/* ijk */
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        sum = 0.0;
        for (k=0; k<n; k++)
            sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }
}
```

*Variable sum
held in register*

Matrix multiplication (ijk)

```
/* ijk */
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        sum = 0.0;
        for (k=0; k<n; k++)
            sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }
}
```



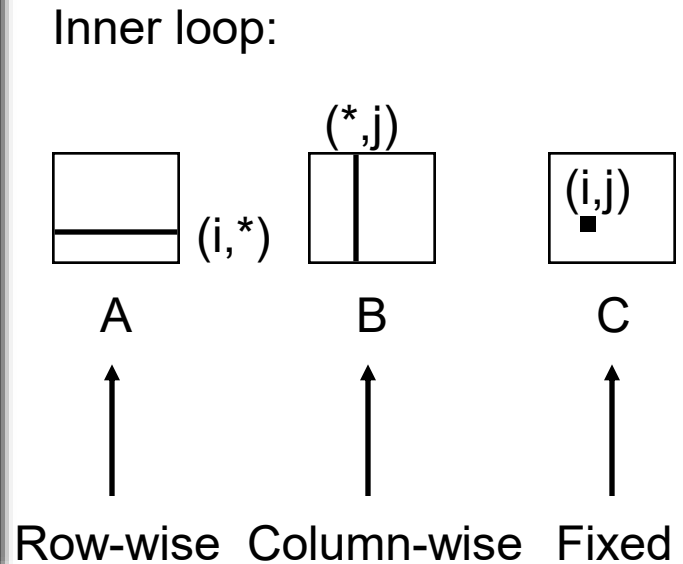
- Misses per Inner Loop Iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.25	1.0	0.0

2 loads, 0 stores
misses/iter = 1.25

Matrix multiplication (jik)

```
/* jik */
for (j=0; j<n; j++) {
    for (i=0; i<n; i++) {
        sum = 0.0;
        for (k=0; k<n; k++)
            sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }
}
```



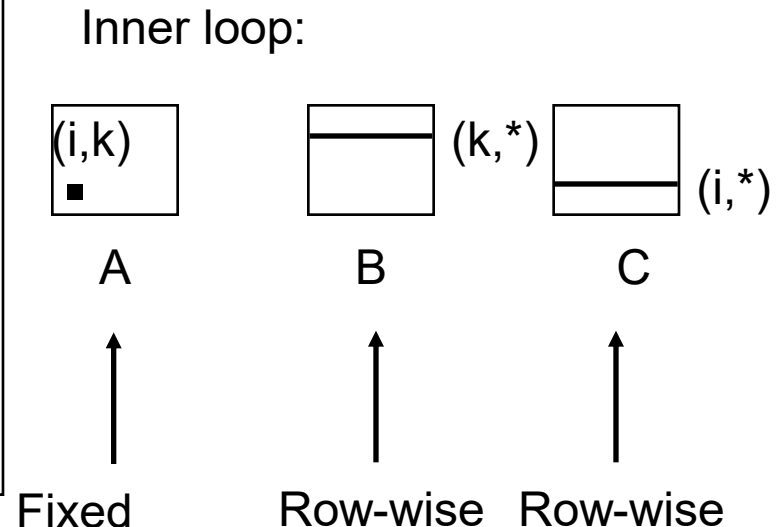
- Misses per Inner Loop Iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.25	1.0	0.0

2 loads, 0 stores
misses/iter = 1.25

Matrix multiplication (kij)

```
/* kij */
for (k=0; k<n; k++) {
    for (i=0; i<n; i++) {
        r = a[i][k];
        for (j=0; j<n; j++)
            c[i][j] += r * b[k][j];
    }
}
```



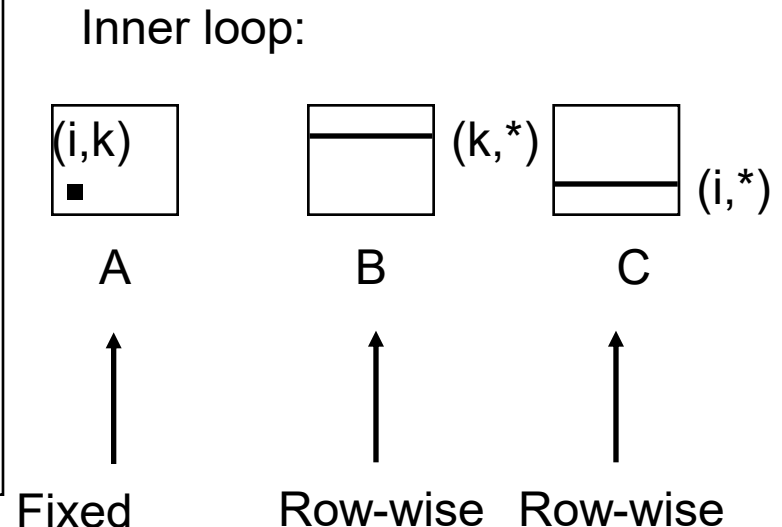
- Misses per Inner Loop Iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

2 loads, 1 store
misses/iter = 0.5

Matrix multiplication (ikj)

```
/* ikj */
for (i=0; i<n; i++) {
    for (k=0; k<n; k++) {
        r = a[i][k];
        for (j=0; j<n; j++)
            c[i][j] += r * b[k][j];
    }
}
```



- Misses per Inner Loop Iteration:

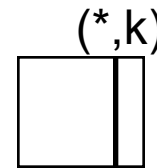
<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

2 loads, 1 store
misses/iter = 0.5

Matrix multiplication (jki)

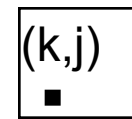
```
/* jki */
for (j=0; j<n; j++) {
    for (k=0; k<n; k++) {
        r = b[k][j];
        for (i=0; i<n; i++)
            c[i][j] += a[i][k] * r;
    }
}
```

Inner loop:



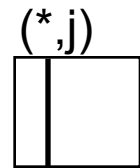
A

Column-wise



B

Fixed



C

Column-wise

Misses per Inner Loop Iteration:

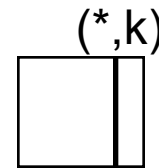
<u>A</u>	<u>B</u>	<u>C</u>
1.0	0.0	1.0

2 loads, 1 store
misses/iter = 2.0

Matrix multiplication (kji)

```
/* kji */
for (k=0; k<n; k++) {
    for (j=0; j<n; j++) {
        r = b[k][j];
        for (i=0; i<n; i++)
            c[i][j] += a[i][k] * r;
    }
}
```

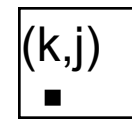
Inner loop:



A



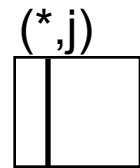
Column-wise



B



Fixed



C

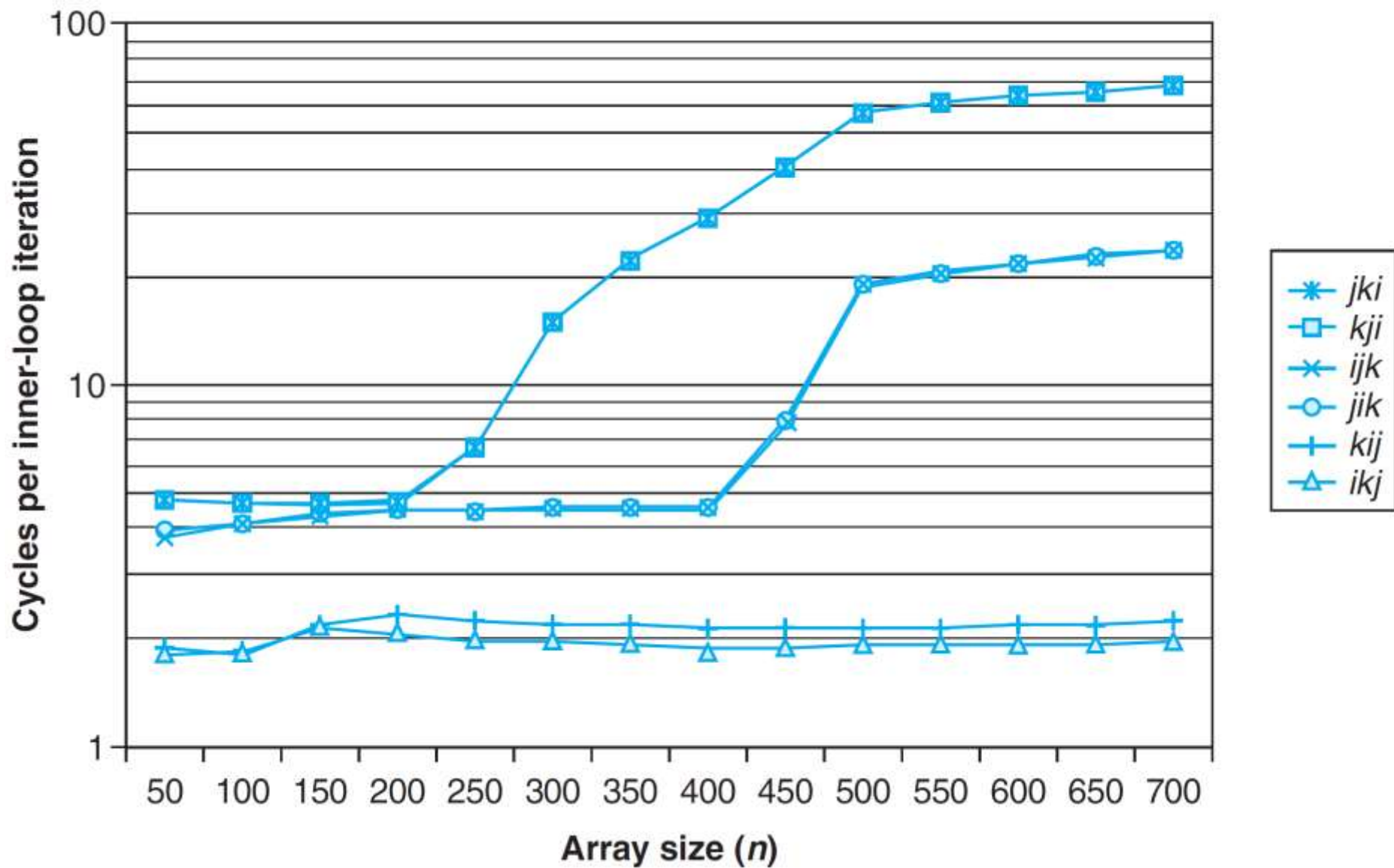


Column-wise

Misses per Inner Loop Iteration:

<u>A</u>	<u>B</u>	<u>C</u>
1.0	0.0	1.0

2 loads, 1 store
misses/iter = 2.0



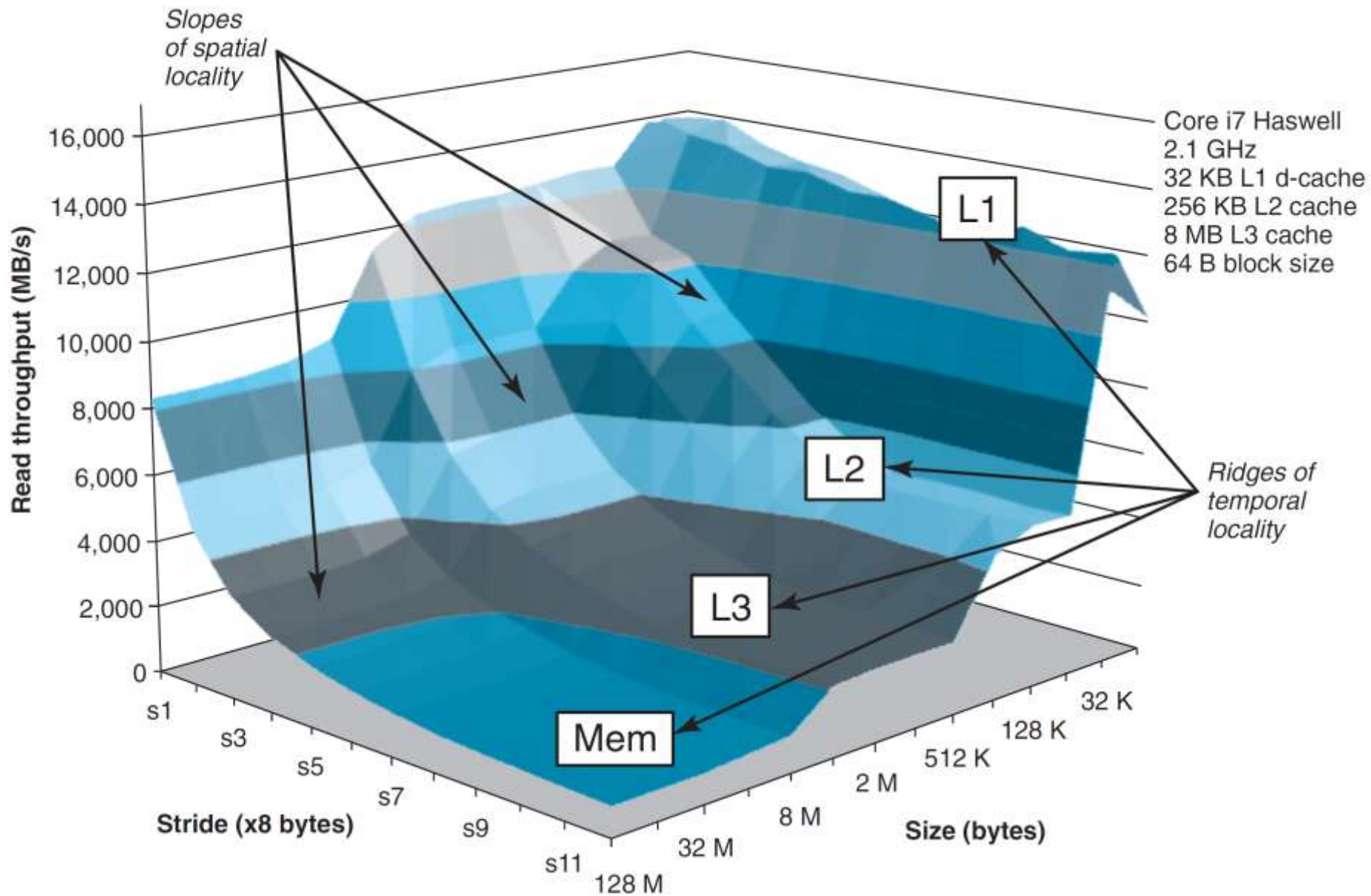
Pentium matrix multiply performance

- The performance difference is
 - almost 40 times for the same application
- Pairs of versions have almost identical measured performance
 - with the same number of memory references and misses per iteration
- The worst memory behavior versions run significantly slower than the other versions
 - in terms of the number of accesses and misses per iteration

Pentium matrix multiply performance

- Miss rate, in this case, is a better predictor of performance than the total number of memory accesses.
- The performance of the fastest pair of versions (kij and ikj) is constant
 - the array is much larger than any of the cache
- The prefetching hardware is
 - smart enough to recognize the stride-1 access
 - fast enough to keep up with memory accesses in the tight inner loop

Memory Mountain



The Memory Mountain

- Read throughput (read bandwidth)
 - The rate that a program reads data from the memory system
- Memory mountain
 - A two-dimensional function of read bandwidth versus temporal and spatial locality
 - Characterizes the capabilities of the memory system for each computer

Memory mountain main routine

/* mountain.c - Generate the memory mountain. */

#define MINBYTES (1 << 14) /* Working set size ranges from 16 KB */

#define MAXBYTES (1 << 27) /* ... up to 128 MB */

#define MAXSTRIDE 12 /* Strides range from 1 to 12 */

#define MAXELEMS MAXBYTES/sizeof(long)

long data[MAXELEMS]; /* The array we'll be traversing */

Memory mountain main routine

```
int main()
{
    int size;                /* Working set size (in bytes) */
    int stride;              /* Stride (in array elements) */
    double Mhz;              /* Clock frequency */
    init_data(data, MAXELEMS); /* Initialize each element in data to 1 */
    Mhz = mhz(0);            /* Estimate the clock frequency */
}
```

Memory mountain main routine

```
for (size = MAXBYTES; size >= MINBYTES; size >>= 1) {
```

```
    for (stride = 1; stride <= MAXSTRIDE; stride++)
```

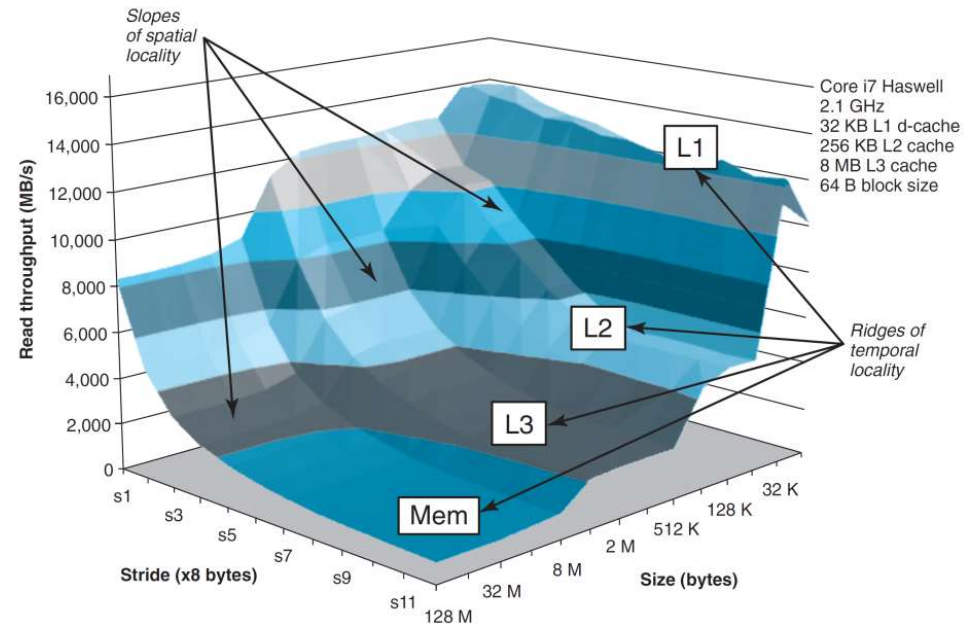
```
        printf("%.1ft", run(size, stride, Mhz));
```

```
        printf("\n");
```

```
}
```

```
exit(0);
```

```
}
```



Memory mountain test function

```
/* Run test (elems, stride) and return read throughput (MB/s) */  
double run (int size, int stride, double Mhz)  
{  
  
    double cycles;  
  
    int elems = size / sizeof(long);  
  
  
    test (elems, stride);                                /* warm up the cache */  
  
    cycles = fcyc2(test, elems, stride, 0);              /* call test (elems, stride) */  
  
    return (size / stride) / (cycles / Mhz);            /* convert cycles to MB/s */  
  
}
```

Memory mountain test function

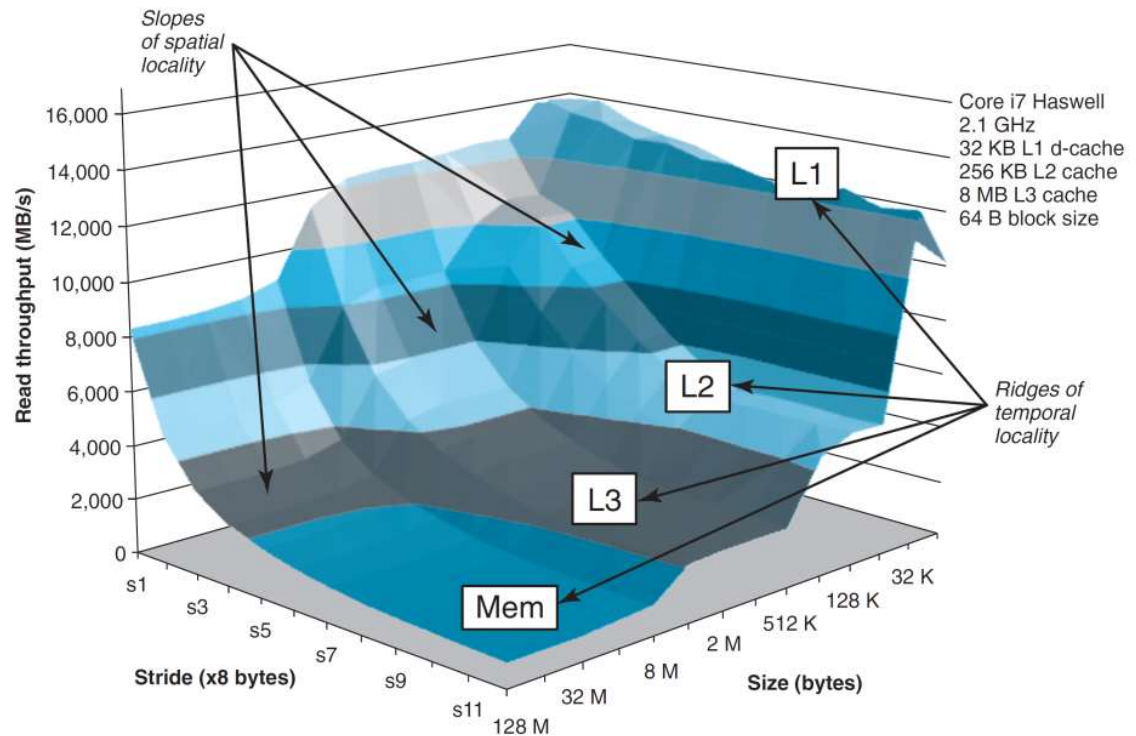
```
/* The test function */  
  
int test (int elems, int stride) {  
  
    long i, sx2=stride*2, sx3=stride*3, sx4=stride*4 ;  
  
    long acc0 = 0, acc1 = 0, acc2 = 0, acc3 =0;  
  
    long length = elems;  
  
    long limit = length – sx4 ;  
  
/*  
  
    for (i = 0; i < elems; i += stride)  
        result += data[i];  
  
*/
```

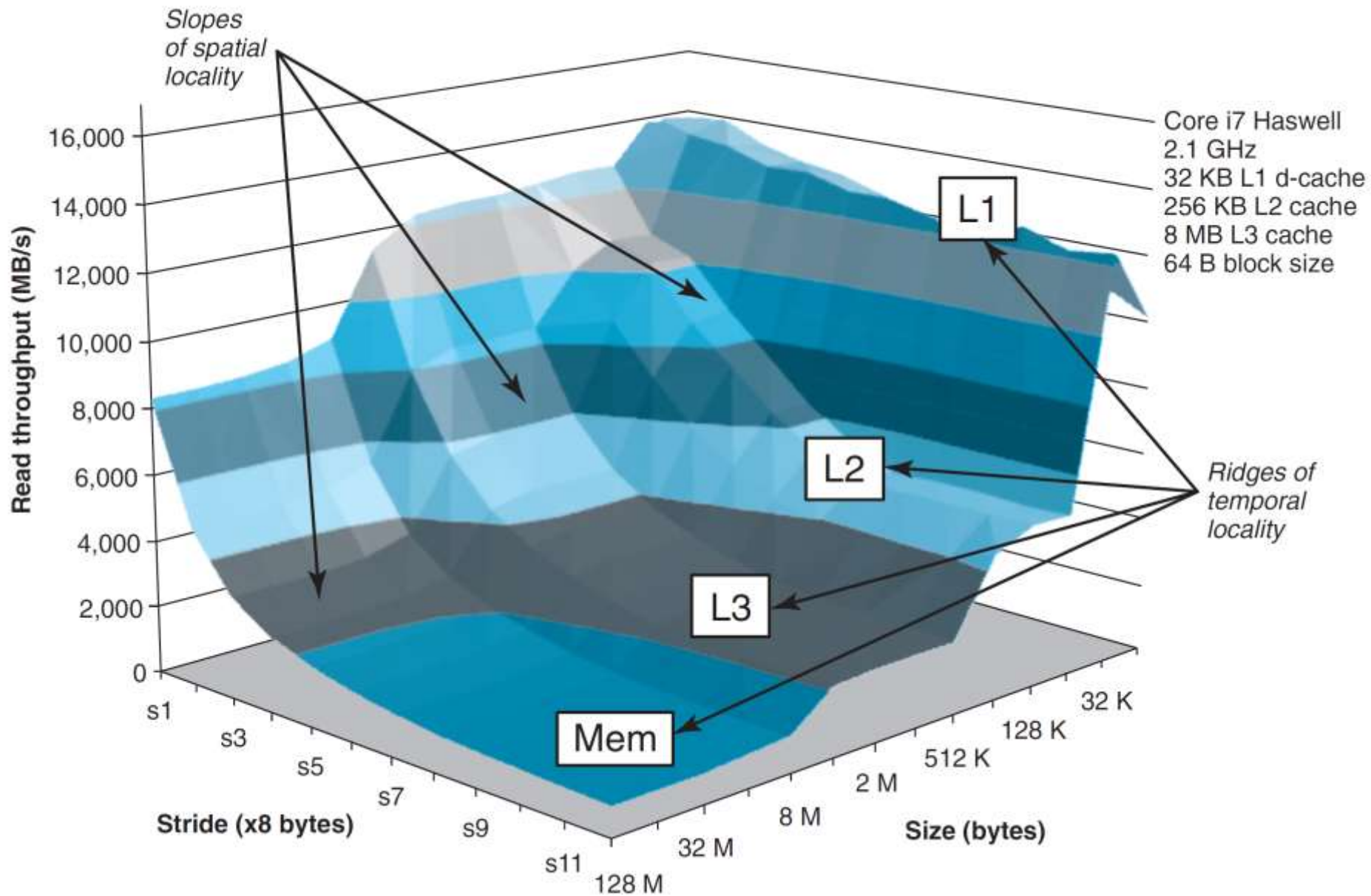
Memory mountain test function

```
for (i = 0; i < limit; i += sx4) {  
    acc0 += data[i];  
    acc1 += data[i+stride];  
    acc2 += data[i+sx2];  
    acc3 += data[i+sx3];  
}  
for (; i < length; i+=stride)  
    acc0 += data[i]  
return ((acc0+acc1)+(acc2+acc3)) ;  
}
```


The Memory Mountain

- Size
 - MAXBYTES(128M) bytes
- Partially accessed
 - Working set: from 128MB to 16KB
 - Stride: from 1 to 16

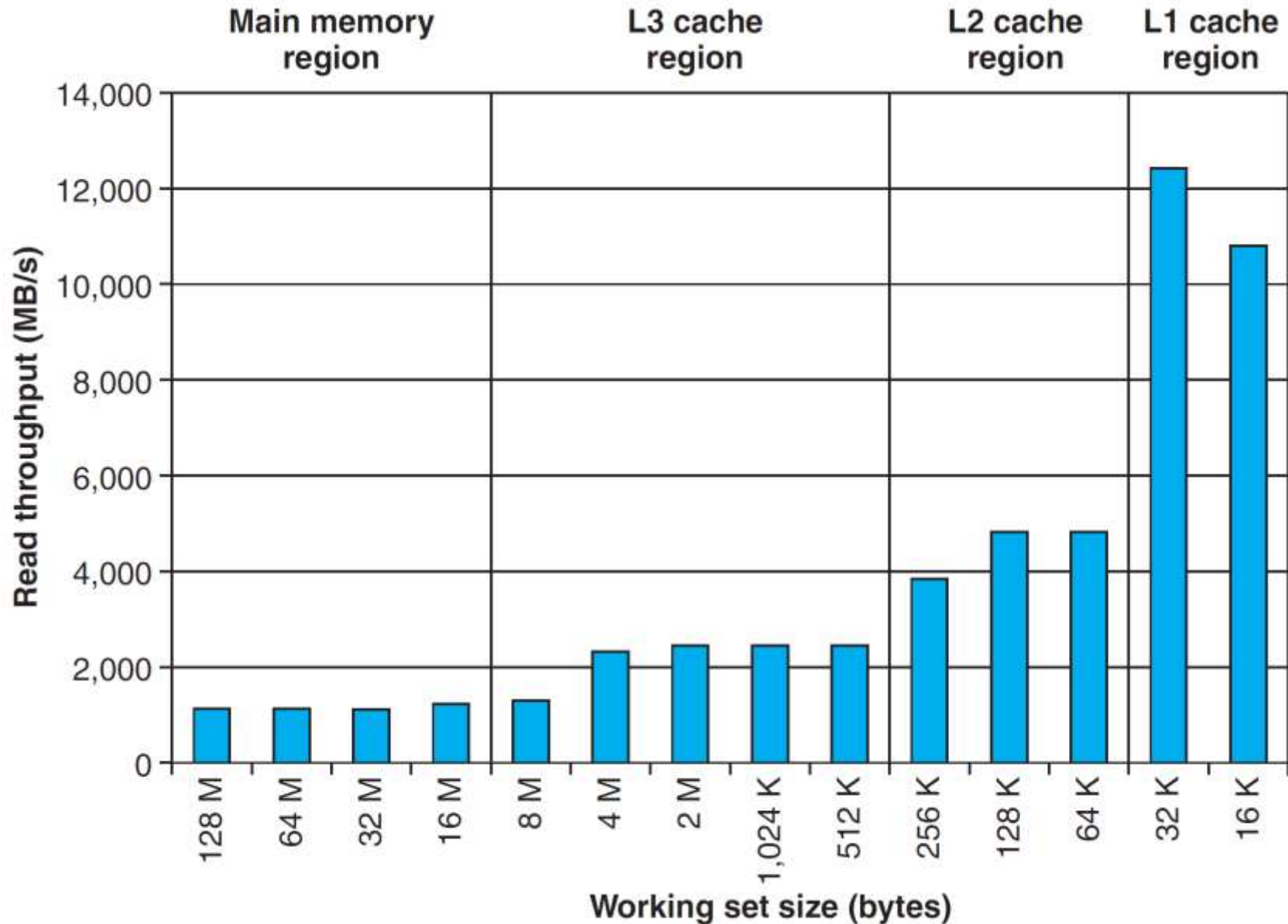




Ridges of temporal locality

- Slice through the memory mountain with `stride=8`
 - illuminates read throughputs of different caches and memory

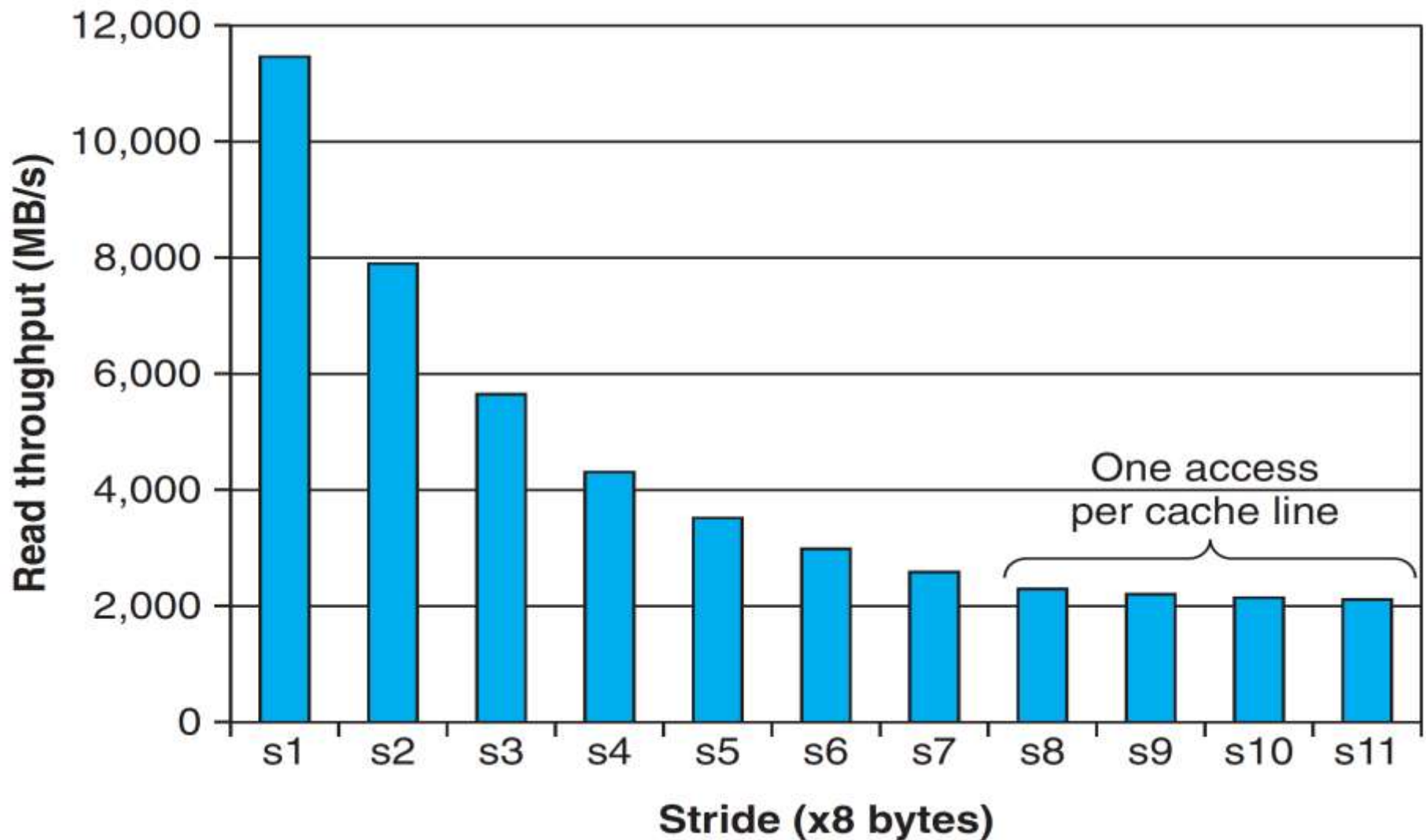
Ridges of temporal locality

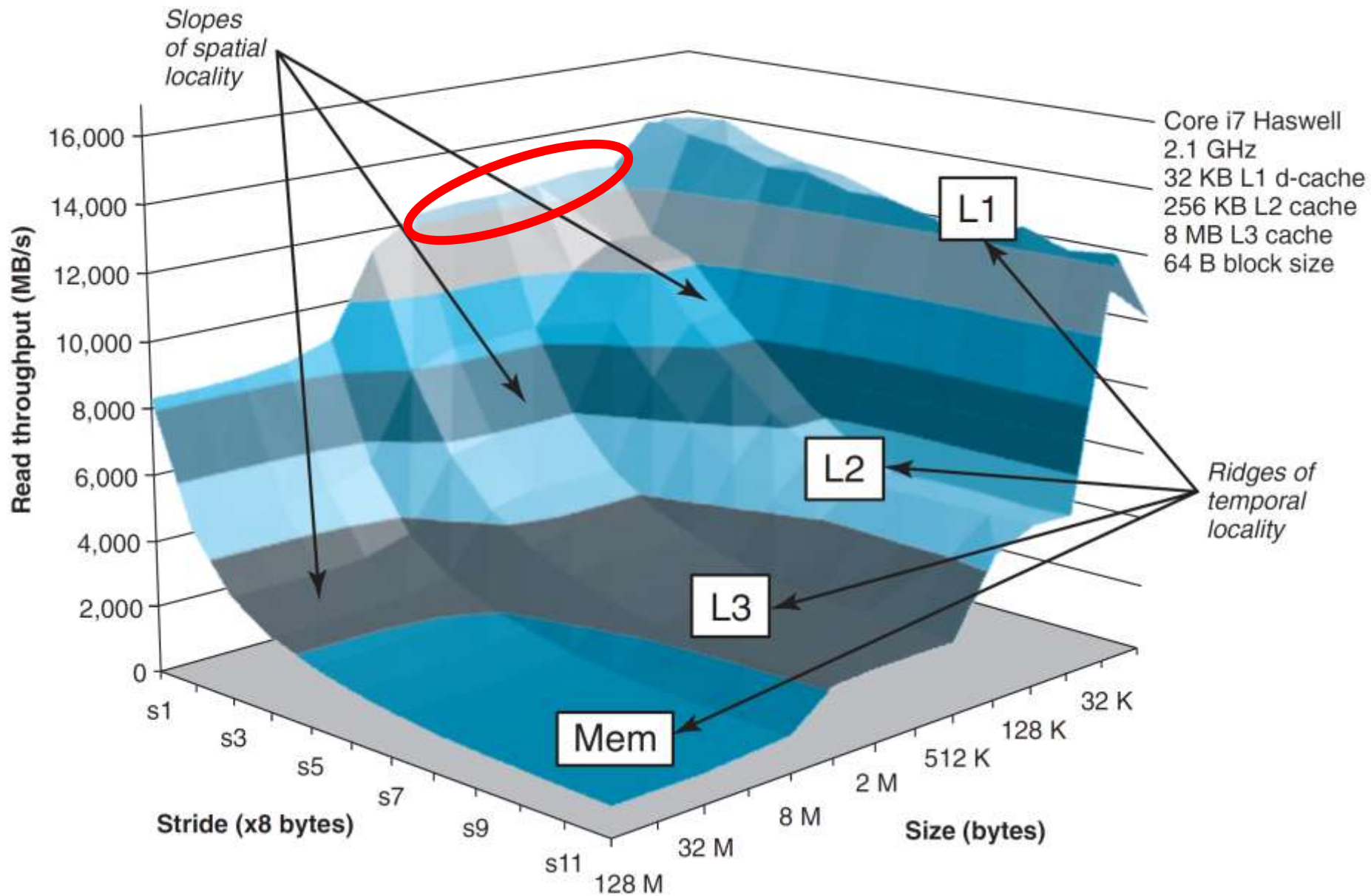


A slope of spatial locality

- Slice through memory mountain with size=4M
 - shows cache block size.

A slope of spatial locality





Interesting Flat Ridge

- Extends perpendicular to the stride axis for a stride of 1
 - The read throughput is a relatively flat 12GB/s even though the working set exceeds the capacities of L1 and L2
 - This is apparently due to a hardware prefetching mechanism